



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251218
Nama Lengkap	Agatha Sabda Alethea Prabawa
Minggu ke / Materi	12 / Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI

Dictionary adalah salah satu tipe data yang tersedia dalam Python dan memiliki kemiripan dengan list, namun dengan cakupan yang lebih luas dan fleksibel. Perbedaan mendasar antara keduanya terletak pada jenis indeks yang digunakan. Pada list, indeks harus berupa bilangan bulat (integer), sedangkan pada dictionary indeks dapat berupa tipe data apa pun, termasuk string, angka, maupun tipe data lainnya.

Setiap anggota dalam dictionary terdiri dari pasangan kunci dan nilai (key-value pair), di mana kunci berfungsi sebagai identitas unik yang tidak boleh ada duplikasinya dalam satu dictionary yang sama. Hubungan antara kunci dan nilai ini disebut juga sebagai item. Secara konseptual, dictionary dapat diartikan sebagai pemetaan dari sekumpulan kunci ke sekumpulan nilai yang bersesuaian.

Sebagai ilustrasi, bayangkan sebuah kamus yang menerjemahkan kata-kata dari bahasa Inggris ke bahasa Spanyol. Setiap kata dalam bahasa Inggris berperan sebagai kunci, dan terjemahannya dalam bahasa Spanyol menjadi nilainya. Untuk membuat dictionary kosong, Python menyediakan fungsi bawaan `dict()`. Perlu diperhatikan bahwa karena `dict` adalah nama fungsi bawaan Python, penggunaannya sebagai nama variabel sebaiknya dihindari agar tidak terjadi konflik.

Penambahan item ke dalam dictionary dilakukan menggunakan tanda kurung kotak dengan format `dictionary[kunci] = nilai`. Untuk menampilkan seluruh isi dictionary, cukup gunakan fungsi `print()`. Tanda kurung kurawal `{}` digunakan untuk merepresentasikan dictionary, baik yang kosong maupun yang sudah berisi data, misalnya `{'one': 'uno', 'two': 'dos'}`.

Perlu diketahui bahwa urutan tampilan item dalam dictionary tidak selalu sesuai dengan urutan saat item dimasukkan. Hal ini tidak menjadi masalah karena pengaksesan nilai dalam dictionary dilakukan melalui kunci, bukan melalui posisi indeks seperti pada list. Apabila kunci yang diminta tidak ditemukan dalam dictionary, Python akan mengeluarkan pesan kesalahan bertipe `KeyError`.

Python juga menyediakan beberapa fungsi dan operator yang dapat digunakan bersama dictionary. Fungsi `len()` mengembalikan jumlah pasangan kunci-nilai yang ada. Operator `in` dapat digunakan untuk memeriksa apakah suatu kunci terdapat dalam dictionary dan akan mengembalikan nilai `True` atau `False`. Untuk memeriksa apakah suatu nilai (bukan kunci) terdapat dalam dictionary, dapat digunakan method `values()` yang menghasilkan kumpulan nilai, kemudian dikonversi ke list sebelum diperiksa dengan operator `in`.

Dari sisi efisiensi, pencarian dalam dictionary jauh lebih cepat dibandingkan pencarian dalam list. List menggunakan algoritma pencarian linear yang waktu pencariannya akan meningkat seiring bertambahnya jumlah elemen. Sementara itu, dictionary menggunakan algoritma Hash Table yang memungkinkan waktu pencarian tetap konstan meskipun jumlah data terus bertambah.

Dictionary sebagai set penghitung (counters)

Salah satu pemanfaatan dictionary yang sangat praktis adalah sebagai alat penghitung kemunculan suatu karakter atau kata. Sebagai contoh, apabila diberikan sebuah string dan kita ingin mengetahui berapa kali setiap huruf muncul di dalamnya, terdapat beberapa pendekatan yang bisa digunakan:

1. Membuat 26 variabel terpisah untuk setiap huruf alfabet, kemudian memeriksa setiap karakter dalam string dan menambah hitungan pada variabel yang sesuai menggunakan kondisi berantai.
2. Membuat list dengan 26 elemen, mengonversi setiap karakter menjadi angka menggunakan fungsi bawaan, lalu menggunakan angka tersebut sebagai indeks untuk menambah hitungan.

3. Membuat dictionary dengan karakter sebagai kunci dan jumlah kemunculannya sebagai nilai, kemudian menambah atau memperbarui nilai setiap kali karakter tersebut ditemukan.

Dari ketiga pendekatan tersebut, penggunaan dictionary dinilai paling praktis karena tidak perlu mengetahui terlebih dahulu huruf apa saja yang akan muncul dalam string. Dictionary secara otomatis menyediakan ruang hanya untuk huruf-huruf yang benar-benar ada.

Pola komputasi semacam ini dikenal dengan istilah histogram dalam ilmu statistika, yaitu representasi dari kumpulan hitungan atau frekuensi kemunculan suatu elemen. Cara kerjanya adalah dengan melakukan perulangan `for` pada setiap karakter dalam string. Jika karakter tersebut belum ada sebagai kunci dalam dictionary, maka dibuat entri baru dengan nilai awal 1. Jika sudah ada, maka nilainya ditambah 1.

Dictionary juga memiliki method `get()` yang dapat dimanfaatkan untuk menyederhanakan proses penghitungan ini. Method `get()` menerima dua argumen: kunci yang dicari dan nilai default yang akan dikembalikan jika kunci tersebut tidak ditemukan. Dengan menggunakan `get()`, pengecekan kondisi `if-else` tidak lagi diperlukan sehingga kode menjadi jauh lebih ringkas. Pola penulisan `d[c] = d.get(c, 0) + 1` telah menjadi idiom umum yang banyak digunakan dalam pemrograman Python.

Dictionary dan File

Kegunaan lain dari dictionary yang tidak kalah penting adalah untuk menghitung frekuensi kemunculan kata-kata dalam sebuah file teks. Pendekatan ini sangat berguna dalam analisis teks, seperti pengolahan artikel, berita, atau karya sastra.

Proses dasarnya dimulai dengan membuka file teks menggunakan blok `try-except` untuk menangani kemungkinan file tidak ditemukan. Setelah file berhasil dibuka, program membaca setiap baris satu per satu menggunakan perulangan `for`. Setiap baris kemudian dipecah menjadi daftar kata menggunakan method `split()`, dan setiap kata diperiksa apakah sudah ada dalam dictionary.

Pola yang digunakan dalam proses ini melibatkan dua perulangan bersarang (nested loop). Perulangan luar bertugas membaca setiap baris dalam file, sementara perulangan dalam memproses setiap kata dalam baris tersebut. Perulangan dalam berjalan lebih cepat karena dieksekusi untuk setiap iterasi perulangan luar. Kombinasi keduanya memastikan bahwa setiap kata dalam seluruh file terhitung dengan benar.

Hasil akhirnya adalah sebuah dictionary yang memetakan setiap kata ke jumlah kemunculannya dalam file. Namun perlu dicatat bahwa urutan tampilan data dalam dictionary tidak teratur secara alfabetis karena Python menyimpannya berdasarkan algoritma hash, bukan berdasarkan urutan alfabet.

Looping dan Dictionary

Perulangan for dapat digunakan secara langsung pada dictionary. Ketika perulangan for diterapkan pada dictionary, Python secara otomatis akan menelusuri seluruh kunci yang ada di dalamnya satu per satu. Untuk mengambil nilai yang bersesuaian dengan setiap kunci, digunakan operator indeks seperti `dictionary[kunci]`.

Pendekatan ini sangat fleksibel dan dapat dikombinasikan dengan kondisi tertentu. Misalnya, jika hanya ingin menampilkan pasangan kunci-nilai yang nilainya melebihi angka tertentu, cukup tambahkan pernyataan if di dalam perulangan tersebut. Dengan cara ini, program hanya akan mencetak entri-entri yang memenuhi kriteria yang ditentukan.

Apabila diperlukan tampilan data yang teratur secara alfabetis berdasarkan kunci, Python menyediakan cara yang cukup mudah. Langkah pertama adalah mengambil seluruh kunci dari dictionary menggunakan method `keys()` dan mengonversinya menjadi list. Selanjutnya, list tersebut diurutkan menggunakan method `sort()`. Terakhir, dilakukan perulangan pada list yang sudah teratur tersebut sambil mengakses nilai yang bersesuaian dari dictionary.

Dengan pendekatan ini, keluaran program akan menampilkan pasangan kunci-nilai dalam urutan alfabet yang rapi meskipun dictionary itu sendiri tidak menyimpan datanya dalam urutan tertentu. Hal ini menunjukkan fleksibilitas dictionary dalam bekerja sama dengan struktur data lain seperti list.

Advanced Text Parsing

Pada pembahasan sebelumnya, contoh file yang digunakan sudah dalam kondisi bersih tanpa tanda baca. Namun dalam situasi nyata, teks biasanya mengandung berbagai tanda baca seperti koma, titik, tanda seru, dan lain sebagainya. Hal ini menimbulkan permasalahan tersendiri karena Python akan menganggap kata yang diikuti tanda baca sebagai kata yang berbeda dari kata yang sama tanpa tanda baca. Misalnya, kata "soft" dan "soft!" akan dihitung sebagai dua kata yang berbeda dan disimpan sebagai dua entri terpisah dalam dictionary.

Selain masalah tanda baca, perbedaan huruf kapital juga menjadi kendala. Kata "Who" dan "who" dianggap sebagai dua kata yang berbeda karena Python bersifat case-sensitive. Akibatnya, frekuensi kemunculan kata tersebut tidak terhitung dengan benar jika tidak ada penanganan khusus.

Untuk mengatasi kedua masalah tersebut, Python menyediakan beberapa solusi yang dapat dikombinasikan. Pertama, method `lower()` digunakan untuk mengubah seluruh teks menjadi huruf kecil sehingga perbedaan kapitalisasi tidak lagi menjadi masalah. Kedua, untuk menghapus tanda baca, Python menyediakan method `translate()` yang bekerja bersama method `str.maketrans()`.

Method `translate()` bekerja dengan cara mengganti karakter tertentu sesuai pemetaan yang diberikan, atau menghapus karakter yang masuk dalam daftar penghapusan. Untuk keperluan menghapus tanda baca, Python menyediakan modul `string` yang di dalamnya terdapat atribut `string.punctuation` berisi seluruh karakter tanda baca standar. Dengan mengombinasikan `translate()` dan `string.punctuation`, semua tanda baca dalam teks dapat dihapus secara otomatis dalam satu langkah.

Urutan pemrosesan yang dianjurkan dalam parsing teks tingkat lanjut ini adalah sebagai berikut:

4. Membaca setiap baris dari file teks.
5. Membersihkan spasi di ujung baris menggunakan method `rstrip()`.
6. Menghapus seluruh tanda baca menggunakan `translate()` dan `string.punctuation`.

7. Mengubah semua huruf menjadi huruf kecil menggunakan `lower()`.
8. Memecah baris menjadi daftar kata menggunakan `split()`.
9. Menghitung setiap kata ke dalam dictionary.

Dengan menerapkan serangkaian tahapan pemrosesan tersebut, hasil penghitungan kata menjadi jauh lebih akurat karena variasi penulisan yang disebabkan oleh tanda baca maupun perbedaan kapitalisasi sudah ditangani sebelum proses penghitungan dilakukan. Pendekatan ini sangat berguna dalam berbagai aplikasi analisis teks seperti penghitungan frekuensi kata, pembuatan indeks dokumen, maupun analisis sentimen teks sederhana.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

```
1 data = {1:10, 2:20, 3:30, 4:40, 5:50, 6:60}
2
3 print("key value item")
4 for key, value in data.items():
5     print(f'{key}    {value}    {key}')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Use
kAlPro-71251218/minggu12/soal01.py"

key value item

1	10	1
2	20	2
3	30	3
4	40	4
5	50	5
6	60	6

Penjelasan perlangkah soal01:

1. `data = {1:10, 2:20, 3:30, 4:40, 5:50, 6:60}` -> Membuat dictionary dengan 6 pasang data.
2. `print("key value item")` -> Mencetak header table.
3. `for key, value in data.items():` -> Perulangan untuk setiap pasangan key-value.
4. `print(f'{key} {value} {key}')` -> Mencetak key, value, dan key sebagai item.

SOAL 2

```
1  lista = ['red', 'green', 'blue']
2  listb = [█'#ff0000', █'#008000', █'#0000ff']
3
4  hasil = {}
5  for i in range(len(lista)):
6      |  hasil[lista[i]] = listb[i]
7
8  urutan = ['green', 'blue', 'red']
9  hasil1 = {key: hasil[key] for key in urutan}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/leno
ro/PrakAlPro-71251218/minggu12/soal02.py"
{'green': '#008000', 'blue': '#0000ff', 'red': '#ff0000'}
```

Penjelasan perlangkah soal02:

1. lista = ['red', 'green', 'blue'] -> membuat list warna.
2. listb = ['#ff0000', '#008000', '#0000ff'] -> membuat list kode warna.
3. hasil = {} -> inisialisasi dictionary kosong.
4. for i in range(len(lista)): -> Perulangan sebanyak 3 kali.
5. hasil[lista[i]] = listb[i] -> Mengisi dictionary dengan pasangan key-value
6. urutan = ['green', 'blue', 'red'] -> menentukan urutan baru.
7. hasil1 = {key: hasil[key] for key in urutan} -> Dictionary comprehension untuk mengubah urutan.
8. print(hasil1) -> menampilkan hasil akhir.

```

1 nama_file = input("Masukan nama file : ")
2
3 email = {}
4 with open(nama_file, 'r') as f:
5     for baris in f:
6         if baris.startswith('From:'):
7             sender = baris.split()[1]
8             email[sender] = email.get(sender, 0) + 1
9
10 print(email)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> C:\Users\lenovo\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro\PrakAlPro-71251218\minggu12\soal03.py"

Masukan nama file : D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro\PrakAlPro-71251218\minggu12\mbox-short.txt

```
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3, 'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1, 'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3, 'david.horwitz@uct.ac.za': 4, 'wagnerm@iupui.edu': 1, 'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2, 'ray@media.berkeley.edu': 1}
```

```
1 nama_file = input("Masukan nama file : ")
2
3 email = {}
4 with open(nama_file, 'r') as f:
5     for baris in f:
6         if baris.startswith('From:'):
7             sender = baris.split()[1]
8             email[sender] = email.get(sender, 0) + 1
9 print(email)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python: soal03

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAIPro> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/PrakAIPro/PrakAIPro-71251218/minggu12/soal03.py"
```

Masukan nama file : D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro\PrakAlPro-71251218\minggu12\mbbox-short.txt

```
{ 'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3, 'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1, 'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3, 'david.horwitz@uct.ac.za': 4, 'wagnermr@iupui.edu': 1, 'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2, 'ray@media.berkeley.edu': 1 }
```

Penjelasan perlangkah soal03:

1. `nama_file = input("Masukan nama file : ")` -> Meminta user mengisi nama file.
2. `email = {}` -> Membuat dictionary kosong untuk menyimpan hasil.
3. `with open(nama_file, 'r') as f:` -> Membuka file dalam mode baca.
4. `for baris in f:` -> Loop membaca setiap baris dalam file.
5. `if baris.startswith('From:')` -> hanya baris yang dimulai dengan 'From:'
6. `sender = baris.split()[1]` -> Memisahkan baris berdasarkan spasi, mengambil elemen ke-2.
7. `email[sender] = email.get(sender, 0) + 1` -> Menghitung jumlah pesan per email.
8. `print(email)` -> Menampilkan dictionary hasil.

SOAL 4

```
1 nama_file = input("Masukan nama file: ")
2
3 domin = {}
4 with open(nama_file, 'r') as f:
5     for baris in f:
6         if baris.startswith('From:'):
7             email = baris.split()[1]
8             domain = email.split('@')[1]
9             domin[domain] = domin.get(domain, 0) + 1
10 print(domin)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python: soal04

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:\Users\lenovo\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/PrakAlPro/PrakAlPro-71251218/minggu12/soal04.py"

Masukan nama file: D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro\PrakAlPro-71251218\minggu12\mbox-short.txt.txt

{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7, 'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}

Penjelasan perlangkah soal04:

1. nama_file = input("MASukan nama file: ") -> Meminta user memasukkan nama file.
2. domin = {} -> Membuat dictionary kosong untuk menyimpan hasil.
3. with open(nama_file, 'r') as f: -> Membuka file dalam mode baca.
4. for baris in f: -> Loop membaca setiap baris dalam file.
5. if baris.startswith('From:'): -> hanya memproses baris yang dimulai dengan 'From:'
6. email = baris.split()[1] -> Memisahkan baris berdasarkan spasi, lalu mengambil elemen ke-2 (alamat email).
7. domain = email.split('@')[1] -> Memisahkan email dengan @, mengambil bagian setelah @.
8. domin[domain] = domin.get(domain, 0) + 1 -> Menambahkan hitungan untuk domain tersebut.
9. print(domin) -> Menampilkan dictionary hasil.